Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Departamentul Ingineria Software şi Automatică

Report on Laboratory Work No. 8

Process Calls and Pipes Inside xv6

Operating Systems

Done by: st. gr. FAF-241, Caţ Chiril

Checked by: asist. univ. Reiţman Patricia

Chişinău – 2026

Contents

1 Purpose and environment **1**

2 Fork, exec, and wait inside xv6 **1**

3 A two-pipe pingpong **2**

4 Conclusion **2**

1 Purpose and environment

I ran process creation, program replacement, and two-way communication inside xv6. I added `user/run.c` and `user/pingpong.c` to the integrated `os/xv6-riscv` tree and its `UPROGS` list, then built the image and booted it with QEMU on 26 October 2026. Matching code copies are in `lab8/code/`. The required `notes.md` records the Observe answers and command output. The optional `primes` exercise was not part of the required deliverable.

2 Fork, exec, and wait inside xv6

The final `run.c` prints its PID, forks, executes `echo` in the child, and waits for the child's status in the parent:

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

I also compiled and ran two temporary variations. With `exec("ls", args)`, the child printed the xv6 directory listing and exited successfully:

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
... (other directory entries)
console        3 29 0
parent: child 4 ended with status 0
```

With `exec("nosuchprog", args)`, replacement failed, so the child continued at the following error line:

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
```

The xv6 program uses `fork`, `exec`, and `wait`; the Lab 7 POSIX program uses `fork`, `execlp`, and `waitpid`. Both sequences create a child, replace its program, and collect its status. `echo`, rather than the original `run` code, prints `hello from the child` after the successful `exec`. When replacement fails, xv6's program calls `exit(1)`, whereas the Lab 7 program calls `exit(127)`. Those explicit choices explain the different parent-reported statuses.

3 A two-pipe pingpong

`pingpong.c` creates a parent-to-child pipe and a child-to-parent pipe before forking. The parent sends one byte, the child reads it and prints `received ping`, then writes it back. The parent reads the reply and prints `received pong`:

```
$ pingpong
6: received ping
5: received pong
$ pingpong
8: received ping
7: received pong
```

Two pipes are needed because each pipe is a one-way byte stream. One carries the request to the child and the other carries the reply to the parent. Each process closes the ends it does not use. The parent's `read(c2p[0], &b, 1)` blocks until the child writes the reply; this call orders the two messages. The later `wait(&status)` collects the child's exit status.

4 Conclusion

The xv6 run followed the same process lifecycle observed with POSIX calls in Lab 7: `fork` creates a child, `exec` runs a different program in it, and `wait` reaps it. The failed `exec` returned to the child and exposed its explicit exit code. The two-pipe experiment showed how blocking reads coordinate a request and reply without a timing delay.